

04 — Database Specification

Purpose

The purpose of this document is to define the official persistence architecture of Mosaica.

This document translates the approved Domain Model into a database design while preserving business ownership, Aggregate boundaries and data integrity principles.

The Database Specification defines:

- data storage structure,
- table responsibilities,
- relationships,
- constraints,
- indexing strategy,
- persistence rules.

Implementation-specific database technologies are intentionally excluded unless required by architectural decisions.

Database Design Principles

Principle 1 — Domain Alignment

The database structure shall support the approved Domain Model.

Database design shall not redefine business concepts.

Principle 2 — Aggregate Ownership

Each Aggregate Root defines the primary ownership boundary of persisted business data.

Internal Aggregate data shall remain consistent within its boundary.

Principle 3 — No Direct Persistence of Value Objects

Value Objects shall only be persisted according to their ownership context.

A Value Object does not automatically require its own database table.

Principle 4 — Shared Metadata Separation

Shared metadata shall remain separate from User-owned data.

Content information and personal archive information shall never be mixed.

Principle 5 — Avoid Premature Optimization

Database structures shall prioritize clarity and maintainability before optimization.

Performance improvements shall be introduced based on actual requirements.

Database Architecture Overview

The database will be organized around two primary Aggregate persistence boundaries.

Database

Shared Metadata

Content
Person
Genre
Series
Universe

Personal Archive

Library Entry
Collections
Tags
History
Notes

04.01 — Database Architecture

Purpose

The purpose of this section is to define the architectural foundation of the Mosaica persistence layer.

This section establishes how domain concepts are translated into a database structure while preserving:

- Aggregate boundaries,
- ownership rules,
- data integrity,
- scalability,
- maintainability.

Database Architecture defines the overall persistence strategy before individual tables and relationships are designed.

Database Architecture Principles

Principle 1 — Domain First Design

The database structure shall be derived from the approved Domain Model.

The database shall support business concepts rather than determine them.

No database limitation shall redefine the established domain architecture.

Principle 2 — Aggregate-Oriented Persistence

Aggregate boundaries shall guide persistence decisions.

Each Aggregate Root represents the primary entry point for its persisted business data.

Internal Aggregate consistency shall be maintained within the persistence model.

Principle 3 — Separation of Shared and Personal Data

Mosaica contains two fundamentally different data categories.

Shared Metadata

Information describing cultural works.

Examples:

- Content
- Person
- Genre
- Series
- Universe
- Media Type

This data may be referenced by multiple Users.

Personal Archive Data

Information describing an individual User's relationship with cultural works.

Examples:

- Library Entry
- Rating
- Status
- Favorite
- Personal Note
- Consumption History

This data belongs exclusively to the User.

Principle 4 — Relational Integrity

The database shall preserve explicit relationships between business concepts.

Foreign key relationships and constraints shall be used where they improve data integrity.

Principle 5 — Controlled Duplication

Data duplication shall only be introduced when it provides a clear architectural benefit.

The default approach is maintaining a single source of truth.

Database Architecture Decision

Decision

Mosaica shall use a relational database architecture.

Rationale

The decision is based on the following characteristics of the domain.

Structured Relationships

Mosaica contains many explicit relationships:

- Content → Person
- Content → Genre
- Content → Series
- Library Entry → Content
- User → Library Entry
- Collection → Library Entry

A relational model represents these relationships naturally.

Strong Data Integrity Requirements

The system requires consistency for:

- ownership,
- references,
- relationships,
- user data separation.

Relational constraints provide appropriate support for these requirements.

Aggregate Compatibility

The selected architecture supports clear Aggregate persistence.

Library Entry and Content can maintain separate persistence boundaries while preserving their relationships.

Query Requirements

Mosaica requires various query patterns:

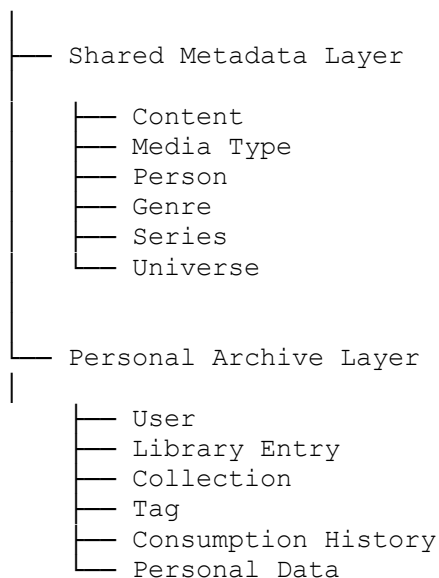
- archive filtering,
- statistics,
- search,
- recommendations,
- metadata browsing.

A relational database provides a strong foundation for these operations.

Database Layer Separation

The database shall be conceptually divided into two logical areas.

Mosaica Database



Aggregate Persistence Model

Content Aggregate

Persistence responsibility:

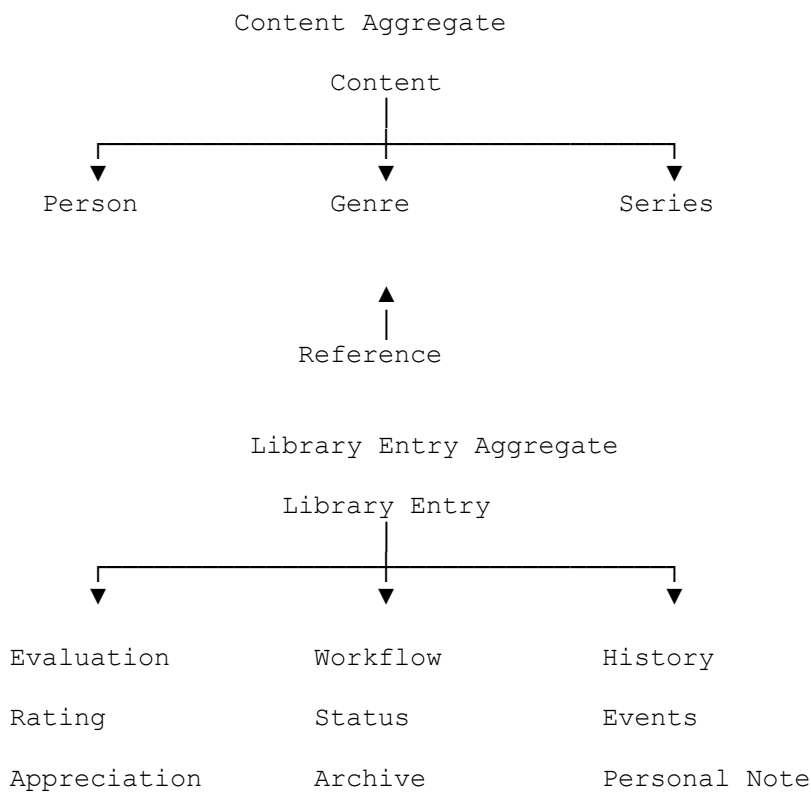
- cultural work information,
- shared metadata relationships,
- media classification.

Library Entry Aggregate

Persistence responsibility:

- User relationship with Content,
 - personal evaluation,
 - workflow state,
 - archive organization,
 - history,
 - personal memory.
-

Persistence Boundary Model



Architecture Constraints

The following constraints are established.

Constraint 1

User-owned information shall never be stored inside Content tables.

Constraint 2

Shared metadata shall never contain User-specific state.

Constraint 3

Library Entry shall remain the central persistence point for personal experience.

Constraint 4

Value Objects shall be persisted according to their owning Aggregate.

Constraint 5

Database structure shall not introduce new domain concepts without architectural approval.

Architectural Significance

The Database Architecture establishes a clean separation between:

- what a cultural work is,
- and what that cultural work means to a specific User.

This separation is one of the core architectural principles of Mosaica.

Shared knowledge remains reusable.

Personal experience remains private and independent.

04.02 — Table Design

Purpose

The purpose of this section is to define the official database table structure of Mosaica.

This section translates the approved Domain Model and Aggregate Design into persistent database structures while preserving:

- ownership boundaries,
- Aggregate responsibilities,
- relationship integrity,
- scalability.

This document defines table responsibilities rather than implementation-specific column definitions.

Table Design Principles

Principle 1 — One Source of Truth

Each business concept shall have a single authoritative storage location.

Duplicate representations of the same business concept shall be avoided unless explicitly justified.

Principle 2 — Aggregate Root Persistence

Every Aggregate Root shall have a dedicated primary table.

The table represents the persistence entry point of the Aggregate.

Principle 3 — Value Object Persistence

Value Objects shall be persisted according to their complexity and ownership.

A Value Object does not automatically require an independent table.

Principle 4 — Relationship Tables

Many-to-many relationships shall be represented through dedicated relationship tables.

Relationship tables shall not introduce independent business concepts unless required.

Database Table Overview

The initial database structure is divided into three categories.

1. Aggregate Tables

These tables represent Aggregate Roots.

Table	Aggregate
users	User
contents	Content
library_entries	Library Entry

users

Responsibility

Stores identity and ownership information.

Domain Responsibility

Represents the User who owns personal archive data.

contents

Responsibility

Stores shared cultural work information.

Domain Responsibility

Represents the Content Aggregate Root.

library_entries

Responsibility

Stores the User's personal relationship with Content.

Domain Responsibility

Represents the Library Entry Aggregate Root.

2. Supporting Entity Tables

These tables represent independent Domain Entities that are not Aggregate Roots.

Table	Entity
collections	Collection
tags	Tag
persons	Person
genres	Genre
series	Series
universes	Universe

collections

Responsibility

Stores User-created organizational groups.

Collections organize Library Entries but do not own them.

tags

Responsibility

Stores User-defined classification labels.

Tags provide flexible organization without changing the identity of Library Entries.

persons

Responsibility

Stores shared contributor information.

Examples:

- actors,
- directors,
- authors,
- creators.

genres

Responsibility

Stores shared classification information.

series

Responsibility

Stores shared series relationships.

universes

Responsibility

Stores shared fictional or conceptual universes.

3. Relationship Tables

Relationship tables represent connections between concepts.

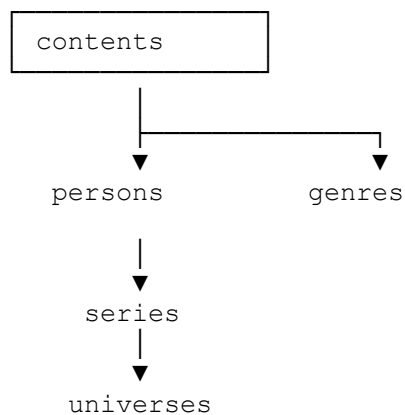
They do not represent independent business objects.

Table	Relationship
content_persons	Content ↔ Person
content_genres	Content ↔ Genre
collection_entries	Collection ↔ Library Entry
entry_tags	Library Entry ↔ Tag

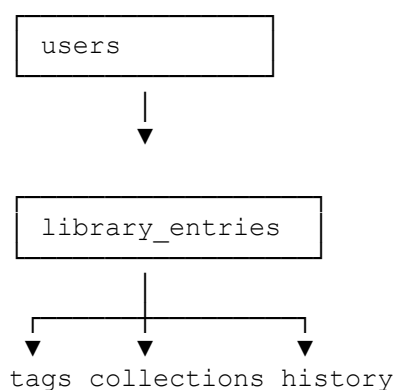
Initial Table Architecture

Mosaica Database

Shared Metadata



Personal Archive



Value Object Persistence Mapping

The following Value Objects do not receive independent tables.

They are stored within their owning Aggregate structures.

Library Entry Value Objects

Stored within `library_entries`:

- Rating
 - Appreciation Level
 - Status
 - Archive Location
 - Favorite
 - Personal Note
-

History Persistence

Consumption History is treated differently.

Because it represents multiple historical records:

- Consumption Event → separate table
 - Consumption Counter → derived value
-

Consumption History Tables

Table	Purpose
<code>consumption_events</code>	Stores individual experiences

The Consumption Counter is calculated from Consumption Events.

It shall not be treated as the primary source of historical truth.

Updated Table Overview

Aggregate Tables

`users`
`contents`
`library_entries`

Supporting Tables

collections
tags
persons
genres
series
universes

Relationship Tables

content_persons
content_genres
collection_entries
entry_tags

History Tables

consumption_events

Architectural Decisions Confirmed

Decision 01

Aggregate Roots receive dedicated persistence tables.

Decision 02

Library Entry Value Objects remain inside the Library Entry persistence boundary.

Decision 03

Consumption Events are persisted independently because they represent multiple historical records.

Decision 04

Consumption Counter is derived information and does not require primary persistence.

Decision 05

Relationship tables represent connections, not new Domain concepts.

04.03 — Entity Persistence Mapping

Purpose

The purpose of this section is to define the relationship between Mosaica Domain concepts and their database persistence representation.

This mapping ensures that the database structure remains aligned with the approved Domain Model.

The objective is not to duplicate the Domain Model inside the database, but to establish clear persistence responsibilities for each business concept.

Persistence Mapping Principles

Principle 1 — Domain Ownership Determines Persistence Ownership

The database representation of a concept shall follow its domain ownership.

Data shall be stored according to the Aggregate and Entity responsible for maintaining its consistency.

Principle 2 — Persistence Does Not Change Domain Responsibility

A database table may contain multiple persisted attributes, but it shall not introduce new business responsibilities.

The database supports the Domain Model.

It does not redefine it.

Principle 3 — Internal Aggregate Data Remains Encapsulated

Data belonging to an Aggregate shall be persisted within that Aggregate's persistence boundary unless there is a strong architectural reason for separation.

Aggregate Persistence Mapping

User Entity

Domain Concept

User

Persistence Representation

`users`

Responsibility

Stores identity and ownership information.

Stored Information

The table represents:

- user identity,
 - account-related information,
 - ownership context.
-

Does Not Store

The User table does not store:

- Library Entries,
- Ratings,
- Collections,

- Tags,
- Personal Notes.

These remain separate concepts.

Content Aggregate

Domain Concept

Content

Persistence Representation

contents

Responsibility

Stores shared cultural work information.

Stored Information

The Content persistence boundary contains:

- cultural work identity,
 - descriptive metadata,
 - Media Type reference,
 - shared metadata relationships.
-

Related Persistence Structures

Domain Concept	Persistence
Media Type	Column / reference
Person	Relationship table
Genre	Relationship table
Series	Reference
Universe	Reference

Does Not Store

Content does not store:

- User ratings,
- User status,
- Favorite state,
- Personal Notes,
- Consumption History.

These belong to Library Entry.

Library Entry Aggregate

Domain Concept

Library Entry

Persistence Representation

`library_entries`

Responsibility

Stores the User's personal relationship with Content.

Stored Information

The Library Entry persistence boundary contains:

Identity

- Library Entry identity
- User reference
- Content reference

Evaluation State

- Rating
 - Appreciation Level
-

Workflow State

- Status
-

Archive State

- Archive Location
 - Favorite
-

Personal Memory

- Personal Note
-

Does Not Store

Library Entry does not duplicate Content metadata.

It references Content instead.

Value Object Persistence Mapping

Rating

Domain Type

Value Object

Persistence Strategy

Stored within `library_entries`.

Reason

Rating has meaning only inside a Library Entry.

Appreciation Level

Domain Type

Value Object

Persistence Strategy

Stored within `library_entries`.

Reason

Appreciation represents User evaluation of a specific Library Entry.

Status

Domain Type

Value Object

Persistence Strategy

Stored within `library_entries`.

Reason

Status represents the current workflow state of a Library Entry.

Archive Location

Domain Type

Value Object

Persistence Strategy

Stored within `library_entries`.

Reason

Archive organization belongs to the Library Entry lifecycle.

Favorite

Domain Type

Value Object

Persistence Strategy

Stored within `library_entries`.

Reason

Favorite represents a personal preference attached to a Library Entry.

Personal Note

Domain Type

Value Object

Persistence Strategy

Stored within `library_entries`.

Reason

Personal Notes have no independent identity.

Consumption History Mapping

Domain Concept

Consumption Event

Persistence Representation

`consumption_events`

Responsibility

Stores individual historical interactions between User and Content.

Reason for Separate Persistence

Unlike other Value Objects, Consumption Event represents multiple historical records.

A single Library Entry may contain many Consumption Events.

Therefore, independent storage is required.

Supporting Entity Mapping

Collection

Persistence Representation

`collections`

Responsibility

Stores User-created organizational groups.

Relationship

Collection connects to Library Entry through:

`collection_entries`

Tag

Persistence Representation

`tags`

Responsibility

Stores User-defined labels.

Relationship

Tag connects to Library Entry through:

entry_tags

Shared Metadata Entities

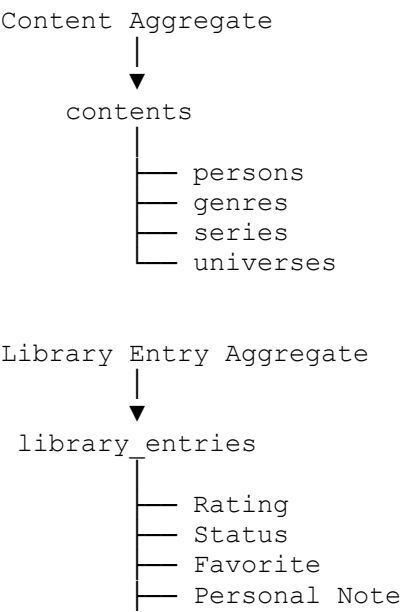
Domain Entity	Table
Person	persons
Genre	genres
Series	series
Universe	universes

These entities belong to the shared metadata layer.

Persistence Overview

Domain Model

Persistence Model



└─ Appreciation Level



consumption_events

Architectural Decisions Confirmed

Decision 01

Content and Library Entry remain separate persistence boundaries.

Decision 02

User-owned information is never stored in shared metadata tables.

Decision 03

Most Library Entry Value Objects are stored inside the Library Entry table.

Decision 04

Consumption Events require separate persistence because they represent historical collections.

Decision 05

Relationship tables represent associations rather than independent business concepts.

04.04 — Relationship Design

Purpose

The purpose of this section is to define the official relationship architecture between Mosaica database entities.

This section establishes:

- relationship types,
- foreign key ownership,
- cardinality,
- relationship responsibilities,
- integrity rules.

The goal is to preserve the approved Domain Model within the persistence layer.

Relationship Design Principles

Principle 1 — Relationships Follow Domain Meaning

Database relationships shall represent business relationships defined within the Domain Model.

Technical relationships shall not introduce new business concepts.

Principle 2 — Foreign Keys Represent References

A foreign key represents a persistence reference.

It does not automatically represent ownership.

Principle 3 — Aggregate Boundaries Remain Protected

Relationships between Aggregates shall remain references.

They shall not create shared ownership.

Principle 4 — Many-to-Many Relationships Require Explicit Tables

Many-to-many relationships shall be represented through dedicated relationship tables.

These tables shall not become independent Domain Entities unless required.

Core Aggregate Relationships

User → Library Entry

Relationship Type

One-to-Many

Business Meaning

A User may create multiple Library Entries.

Each Library Entry belongs to exactly one User.

Database Representation

Foreign Key:

```
library_entries.user_id  
↓  
users.id
```

Ownership

User owns Library Entries.

However, Library Entry remains its own Aggregate Root.

Cardinality

User

1
|
N

Content → Library Entry

Relationship Type

One-to-Many

Business Meaning

One Content item may be referenced by many Users through their personal Library Entries.

Each Library Entry references exactly one Content.

Database Representation

Foreign Key:

```
library_entries.content_id
      ↓
contents.id
```

Ownership

Content does not own Library Entries.

Library Entries do not own Content.

This is a reference relationship between Aggregates.

Cardinality

Content

1
|
N

Library Entries

Content → Person

Relationship Type

Many-to-Many

Business Meaning

A Content item may have multiple contributors.

A Person may contribute to multiple Content items.

Database Representation

Relationship Table:

`content_persons`

Structure:

`content_id`
`person_id`

Ownership

Neither side owns the other.

The relationship represents shared metadata.

Content → Genre

Relationship Type

Many-to-Many

Business Meaning

A Content item may belong to multiple Genres.

A Genre may classify multiple Content items.

Database Representation

Relationship Table:

`content_genres`

Structure:

`content_id`
`genre_id`

Content → Series

Relationship Type

Many-to-One

Business Meaning

Multiple Content items may belong to the same Series.

A Content item may optionally belong to one Series.

Database Representation

Foreign Key:

`contents.series_id`
↓
`series.id`

Ownership

Series does not own Content.

It provides shared classification structure.

Content → Universe

Relationship Type

Many-to-One

Business Meaning

Multiple Content items may belong to one Universe.

A Content item may optionally belong to one Universe.

Database Representation

Foreign Key:

```
contents.universe_id
      ↓
universes.id
```

Library Entry → Collection

Relationship Type

Many-to-Many

Business Meaning

A Library Entry may belong to multiple Collections.

A Collection may contain multiple Library Entries.

Database Representation

Relationship Table:

`collection_entries`

Structure:

`collection_id`
`library_entry_id`

Ownership

Collections organize Library Entries.

They do not own them.

Library Entry → Tag

Relationship Type

Many-to-Many

Business Meaning

A Library Entry may have multiple Tags.

A Tag may describe multiple Library Entries.

Database Representation

Relationship Table:

`entry_tags`

Structure:

`entry_id`
`tag_id`

Library Entry → Consumption Event

Relationship Type

One-to-Many

Business Meaning

A Library Entry may contain multiple historical Consumption Events.

Each Consumption Event belongs to exactly one Library Entry.

Database Representation

Foreign Key:

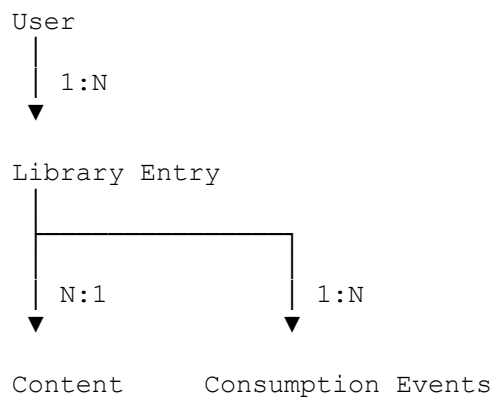
```
consumption_events.library_entry_id
      ↓
library_entries.id
```

Ownership

Consumption Events are owned by the Library Entry Aggregate.

They cannot exist independently.

Relationship Overview



Content

- N:M — Person
- N:M — Genre
- N:1 — Series
- N:1 — Universe

Library Entry

- N:M — Collection
- N:M — Tag

Referential Integrity Rules

Rule 1

A Library Entry cannot exist without a valid User reference.

Rule 2

A Library Entry cannot exist without a valid Content reference.

Rule 3

A Consumption Event cannot exist without a valid Library Entry.

Rule 4

Deleting Content requires explicit handling of related Library Entries.

Cascade behavior shall be defined separately.

Rule 5

Deleting a Collection or Tag shall not delete Library Entries.

Only the relationship shall be removed.

Architectural Decisions Confirmed

Decision 01

Aggregate relationships are represented through references.

Decision 02

Many-to-many relationships use dedicated relationship tables.

Decision 03

Collection and Tag relationships do not create ownership.

Decision 04

Consumption Events remain dependent records of Library Entry.

04.05 — Constraints & Validation

Purpose

The purpose of this section is to define the database-level constraints required to preserve data integrity within the Mosaica persistence layer.

This section establishes structural validation rules while respecting the separation between database responsibilities and domain business logic.

Database constraints protect data consistency.

Business Rules remain responsible for domain behavior.

Constraint Principles

Principle 1 — Database Integrity

The database shall prevent structurally invalid data from being stored.

Principle 2 — Domain Logic Separation

Database constraints shall not replace Domain Rules.

Complex business decisions remain within the application and domain layers.

Principle 3 — Aggregate Consistency

Database structures shall support Aggregate ownership boundaries.

Principle 4 — Explicit Relationships

All required relationships shall be enforced through appropriate constraints.

Primary Key Rules

Every persistent Entity representation shall have a unique identifier.

The following tables require primary keys:

Table	Primary Key
users	user_id
contents	content_id
library_entries	library_entry_id
collections	collection_id
tags	tag_id
persons	person_id
genres	genre_id
series	series_id
universes	universe_id

Table	Primary Key
consumption_events	consumption_event_id

Foreign Key Constraints

Library Entry Ownership

A Library Entry must always belong to a valid User.

Constraint:

```
library_entries.user_id
      ↓
users.user_id
```

Rule:

A Library Entry cannot exist without an owning User.

Library Entry Content Reference

A Library Entry must always reference valid Content.

Constraint:

```
library_entries.content_id
      ↓
contents.content_id
```

Rule:

A Library Entry cannot exist without a Content reference.

Consumption Event Ownership

Every Consumption Event must belong to an existing Library Entry.

Constraint:

```
consumption_events.library_entry_id
      ↓
library_entries.library_entry_id
```

Rule:

Historical records cannot exist independently.

Required Field Constraints

The following concepts are mandatory.

User

Required:

- identity information

A User cannot exist without required identity data.

Content

Required:

- title
- media type

A Content item must always have enough information to be identified within the shared metadata layer.

Library Entry

Required:

- user reference
- content reference
- archive location
- status

A Library Entry must always represent a valid personal archive state.

Unique Constraints

User Identity

User identity fields requiring uniqueness shall be protected.

Example:

```
email UNIQUE
```

Content Identity

Content uniqueness shall be evaluated according to business requirements.

Duplicate Content records shall not be created accidentally.

The final uniqueness strategy shall be defined according to the selected identification approach.

Tag Names

Tags shall maintain uniqueness within the User scope.

Example:

A User cannot create two identical Tags with the same name.

Constraint concept:

```
(user_id, tag_name) UNIQUE
```

Collection Names

Collections shall maintain uniqueness within the User scope.

Example:

A User cannot create multiple Collections with the same name.

Constraint concept:

```
(user_id, collection_name) UNIQUE
```

Relationship Table Constraints

Collection Entries

The relationship table shall prevent duplicate relationships.

Constraint:

```
(collection_id, library_entry_id) UNIQUE
```

Entry Tags

The relationship table shall prevent duplicate tag assignments.

Constraint:

```
(entry_id, tag_id) UNIQUE
```

Content Persons

The relationship table shall prevent duplicate contributor relationships.

Constraint:

```
(content_id, person_id) UNIQUE
```

Content Genres

The relationship table shall prevent duplicate genre relationships.

Constraint:

```
(content_id, genre_id) UNIQUE
```

Value Object Validation Mapping

Rating

Database responsibility:

- store valid rating representation.

Domain responsibility:

- define rating meaning and behavior.

Status

Database responsibility:

- store current status value.

Domain responsibility:

- control allowed status transitions.
-

Archive Location

Database responsibility:

- store current location value.

Domain responsibility:

- control valid archive movement rules.
-

Favorite

Database responsibility:

- store preference state.

Domain responsibility:

- manage preference behavior.
-

Personal Note

Database responsibility:

- store user-generated text.

Domain responsibility:

- manage note lifecycle.

Delete Behavior

Deletion rules shall respect Aggregate boundaries.

Content Deletion

Content deletion requires explicit handling.

Reason:

Library Entries reference Content.

Automatic deletion of personal archive data is not permitted.

User Deletion

User deletion requires explicit handling.

Reason:

User owns personal archive information.

Collection Deletion

Deleting a Collection shall remove relationships only.

Library Entries shall remain unchanged.

Tag Deletion

Deleting a Tag shall remove relationships only.

Library Entries shall remain unchanged.

Consumption Event Deletion

Consumption Events belong to Library Entry history.

Deletion behavior shall preserve historical integrity.

Constraint Summary

Area	Protection
Identity	Primary Keys
Ownership	Foreign Keys
Duplicate Prevention	Unique Constraints
Relationships	Junction Constraints
Required Data	NOT NULL Rules
Historical Integrity	Ownership Constraints

Architectural Decisions Confirmed

Decision 01

Database constraints protect structural integrity.

Decision 02

Business Rules remain outside database constraints.

Decision 03

Aggregate ownership boundaries are preserved through foreign key design.

Decision 04

Relationship deletion shall not accidentally delete unrelated business data.

04.06 — Indexing Strategy

Purpose

The purpose of this section is to define the indexing strategy of the Mosaica database.

Indexes shall be designed according to expected query patterns, relationship frequency and application performance requirements.

The goal is to improve query efficiency while avoiding unnecessary database complexity.

Indexing Principles

Principle 1 — Query Driven Indexing

Indexes shall be created based on actual access patterns.

A column shall not receive an index only because it exists.

Principle 2 — Relationship Optimization

Foreign key columns frequently used for relationship queries shall be considered for indexing.

Principle 3 — User-Centric Performance

Because Mosaica is a personal archive application, user-based queries represent one of the primary performance considerations.

Principle 4 — Avoid Premature Optimization

Indexes shall only be introduced when they provide measurable architectural benefit.

Primary Index Strategy

All Aggregate Root tables shall contain indexed primary keys.

Table	Primary Index
users	user_id
contents	content_id
library_entries	library_entry_id

Primary indexes provide:

- identity lookup,
- relationship targeting,
- database integrity.

User-Based Indexing

Table

`library_entries`

Index Candidate

`user_id`

Reason

The majority of personal archive operations begin from a User context.

Examples:

- "Show my library"
- "Filter my entries"
- "Calculate my statistics"
- "Search my archive"

Decision

Index:

`library_entries(user_id)`

Content Reference Indexing

Table

`library_entries`

Index Candidate

`content_id`

Reason

Content is referenced by multiple Library Entries.

Required queries include:

- finding all Users who have a Content item,
 - checking Content usage,
 - loading Content-related archive data.
-

Decision

Index:

`library_entries(content_id)`

Status Indexing

Table

`library_entries`

Index Candidate

`status`

Reason

Status is a common filtering dimension.

Examples:

- planned items,
- completed works,
- currently active works.

Decision

Index:

```
library_entries(status)
```

Archive Location Indexing

Table

```
library_entries
```

Index Candidate

```
archive_location
```

Reason

Archive navigation frequently depends on location.

Examples:

- Archive
- Wishlist
- Removed items

Decision

Index:

```
library_entries(archive_location)
```

Composite Index Strategy

Some queries require multiple filtering conditions.

For example:

"Show my completed movies."

This query uses:

- User
- Status
- Media Type

Candidate Composite Index

```
library_entries(user_id, status)
```

Decision

Composite indexes shall be introduced according to actual query frequency after implementation analysis.

Version 1 shall prioritize simple predictable indexes.

Relationship Table Indexing

Relationship tables require indexes because they are frequently queried.

Collection Entries

Table:

```
collection_entries
```

Required indexes:

```
(collection_id, library_entry_id)
```

and

```
(library_entry_id, collection_id)
```

Reason

Queries occur in both directions:

- Find entries in a collection.
 - Find collections containing an entry.
-

Entry Tags

Table:

`entry_tags`

Required indexes:

`(entry_id, tag_id)`

and

`(tag_id, entry_id)`

Reason

Queries occur in both directions:

- Find tags of an entry.
 - Find entries using a tag.
-

Content Relationship Indexing

Content Persons

Table:

`content_persons`

Indexes:

`(content_id, person_id)`
`(person_id, content_id)`

Content Genres

Table:

`content_genres`

Indexes:

`(content_id, genre_id)`
`(genre_id, content_id)`

History Indexing

Table

`consumption_events`

Index Candidates

`library_entry_id`

Reason

Most history queries begin from a Library Entry.

Examples:

- Show viewing history.
 - Calculate consumption statistics.
 - Display timeline.
-

Decision

Index:

`consumption_events(library_entry_id)`

Search-Oriented Indexing

Search requirements will likely include:

- Content title,
- Original title,
- Person names,
- Genres.

However, advanced search optimization depends on:

- selected database technology,
- search requirements,
- data volume.

Therefore, dedicated search indexing is deferred.

Index Summary

Table	Index Purpose
library_entries	User archive queries
library_entries	Content lookup
library_entries	Status filtering
library_entries	Archive filtering
consumption_events	History retrieval
relationship tables	Relationship traversal

Architectural Decisions Confirmed

Decision 01

Indexes are created based on query requirements.

Decision 02

Personal archive queries receive priority optimization.

Decision 03

Relationship tables receive bidirectional indexing where navigation requires it.

Decision 04

Advanced search indexing is postponed until implementation requirements are known.

04.07 — Database Architecture Review

Review Objective

The purpose of this Architecture Review is to validate that the Mosaica Database Specification accurately represents the approved Domain Model and Aggregate Design.

This review evaluates:

- persistence boundaries,
- ownership consistency,
- relationship integrity,
- data separation,
- scalability principles.

The objective is to confirm that the database architecture is ready to support future technical design phases.

Scope Reviewed

The following database architecture components have been reviewed.

- Database Architecture
- Table Design
- Entity Persistence Mapping
- Relationship Design
- Constraints & Validation
- Indexing Strategy
- Aggregate Persistence Boundaries

Validation Results

Domain Alignment

Status: Approved

The database architecture remains consistent with the approved Domain Model.

No database structure introduces new business concepts or changes existing responsibilities.

Aggregate Persistence

Status: Approved

Aggregate boundaries are correctly reflected within the persistence architecture.

The two core Aggregates maintain separate persistence responsibilities:

- Library Entry Aggregate
 - Content Aggregate
-

Shared and Personal Data Separation

Status: Approved

Shared metadata and User-owned information remain completely separated.

Shared information:

- Content
- Person
- Genre
- Series
- Universe

Personal information:

- Library Entry
- Rating
- Status
- Favorite
- Personal Note
- Consumption History

are stored within their appropriate boundaries.

Value Object Persistence

Status: Approved

Value Objects are persisted according to their ownership.

Library Entry Value Objects remain within the Library Entry persistence boundary.

Media Type remains within the Content persistence boundary.

No unnecessary Value Object tables have been introduced.

Relationship Integrity

Status: Approved

Relationships accurately represent Domain relationships.

The following principles are preserved:

- References do not imply ownership.
 - Many-to-many relationships use relationship tables.
 - Aggregate boundaries remain protected.
-

Historical Data Management

Status: Approved

Consumption Events are correctly separated from other Value Objects.

Historical records are stored independently because they represent multiple events rather than a single business value.

Consumption Counter remains a derived concept.

Data Integrity

Status: Approved

Database constraints provide structural protection without replacing business logic.

The separation between:

- Database Constraints
- Domain Rules

is preserved.

Performance Strategy

Status: Approved

Indexing decisions follow expected query patterns.

The strategy prioritizes:

1. User archive operations
2. Relationship navigation
3. Historical queries
4. Future search optimization

No unnecessary optimization has been introduced.

Architectural Decisions Confirmed

The following decisions are now official parts of the Mosaica Database Architecture.

Decision 01

Mosaica uses a relational database architecture.

Decision 02

Aggregate Roots define primary persistence boundaries.

Decision 03

Library Entry and Content remain separate persistence domains.

Decision 04

User-owned data is never stored inside shared metadata structures.

Decision 05

Value Objects are persisted according to ownership context.

Decision 06

Consumption Events are stored separately because they represent historical collections.

Decision 07

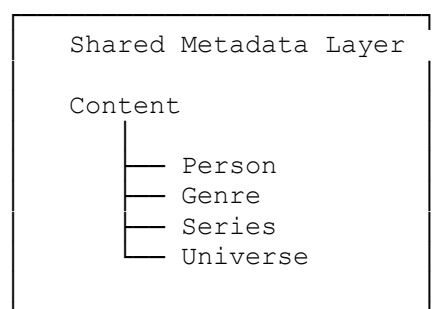
Relationship tables represent associations rather than independent business concepts.

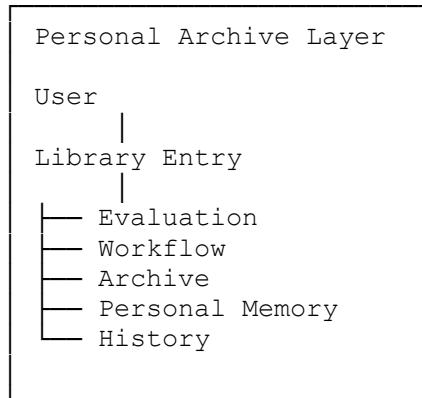
Decision 08

Database optimization follows query requirements rather than assumptions.

Database Architecture Summary

Mosaica Database





Review Conclusion

The Mosaica Database Specification is considered architecturally complete.

The current design successfully translates the approved Domain Model into a persistence architecture while maintaining:

- clear ownership,
- strong data integrity,
- scalable relationships,
- minimal complexity.

No blocking architectural issues have been identified.

The database architecture is approved for use in future technical design phases.